Flex

Smart Contract Security Assessment

May 07, 2026

ABSTRACT

Dedaub was commissioned to perform a security audit of the Flex protocol, a fixed-rate lending protocol inspired by LiquityV2.

BACKGROUND

Flex is a CDP-style lending protocol. Borrowers lock collateral in troves and receive borrow tokens, paying a fixed annual interest rate of their own choosing (bounded, adjustable with a cooldown). Lenders deposit borrow tokens into a Yearn V3 strategy, earning interest and upfront-borrow fees; they also absorb bad debt from underwater liquidations and receive surplus auction proceeds from redemptions. When idle lender liquidity is insufficient – for borrows or for lender withdrawals – the protocol redeems troves and Dutch-auctions their collateral to free up borrow tokens. For borrows, the redeemed troves have strictly lower interest rates than the new borrow, while for lender withdrawals troves are redeemed as necessary, in both cases generally starting from the lowest interest rate trove. Troves can be liquidated by anyone once their collateralization ratio (CR) falls below the minimum (MCR), with a fee that scales linearly between a preset bound based on how far below MCR the trove sits.

SETTING & CAVEATS

This audit report mainly covers the contracts of the repository of the Flex protocol at commit `b4b96567dc4432849f5a8c82a9e51a1b23bf81fe`. The fixes were delivered in the following PR: PR #16.

This report builds on the prior audit reports available in the repository. Previously reported issues that the protocol team has already acknowledged or explicitly decided not to fix are generally not repeated in this report, unless the current review identified a materially new exploit path, an incomplete mitigation, or a distinct impact not covered by the

earlier report. The absence of a previously known and acknowledged issue in this report should not be interpreted as a change in its status or as a reassessment of its validity.

Audit Start Date: April 28, 2026
Report Submission Date: May 07, 2026

Three auditors worked on the following contracts:

```
flex-contracts/src/
├── auction.vy
├── dutch_desk.vy
├── factory.vy
├── lender/
│   ├── LenderFactory.sol
│   ├── Lender.sol
│   └── periphery/
│       └── StrategyAprOracle.sol
├── oracles/
│   ├── yvcrvusd2_to_usdc_oracle.vy
│   ├── yvusd_to_usdc_oracle.vy
│   └── yvweth2_to_usdc_oracle.vy
├── registry.vy
├── sorted_troves.vy
└── trove_manager.vy
```

The audit's main target is security threats, i.e., what the community understanding would likely call "hacking", rather than the regular use of the protocol. Functional correctness (i.e. issues in "regular use") is a secondary consideration. Typically it can only be covered if we are provided with unambiguous (i.e. full-detail) specifications of what is the expected, correct behavior. In terms of functional correctness, we often trusted the code's calculations and interactions, in the absence of any other specification. Functional correctness relative to low-level calculations (including units, scaling and quantities returned from external protocols) is generally most effectively done through thorough testing rather than human auditing.

VULNERABILITIES & FUNCTIONAL ISSUES

This section details issues affecting the functionality of the contract. Dedaub generally categorizes issues according to the following severities, but may also take other considerations into account such as impact or difficulty in exploitation:

Category	Description
CRITICAL	Can be profitably exploited by any knowledgeable third-party attacker to drain a portion of the system's or users' funds OR the contract does not function as intended and severe loss of funds may result.
HIGH	Third-party attackers or faulty functionality may block the system or cause the system or users to lose funds. Important system invariants can be violated.
MEDIUM	Examples: • User or system funds can be lost if external systems misbehave. • DoS, under specific conditions. • Part of the functionality becomes unusable due to a programming error.
LOW	Examples: • Breaking important invariants but without apparent consequences. • Buggy functionality for trusted users where a workaround exists. • Security issues which may manifest when the system evolves.

Issue resolution statuses include "resolved," when auditors verify that the issue has been addressed; "mitigated," when residual risk remains but is materially reduced; "acknowledged," when the client accepts the observation without action; "by design," when the behavior is intentional; and "dismissed," when the client does not consider it an issue.

CRITICAL SEVERITY:

[No critical severity issues]

HIGH SEVERITY:

[No high severity issues]

MEDIUM SEVERITY:

ID	Description	STATUS
M1	`borrow()` on a pre-existing helper Trove bypasses the same-block close guard and suppresses upfront fees	**RESOLVED**

A previously identified issue showed that Flex's upfront fee can be manipulated by temporarily changing the TroveManager-wide average interest rate inside a single transaction.

The upfront fee is calculated from the live values of `total_debt` and `total_weighted_debt`:

`trove_manager::_get_upfront_fee():1393-1402`

```
new_total_debt: uint256 = self.total_debt if is_existing_debt
                          else self.total_debt + debt_amount
new_total_weighted_debt: uint256 =
    self.total_weighted_debt if is_existing_debt else
    self.total_weighted_debt + (debt_amount * annual_interest_rate)

avg_interest_rate: uint256 = new_total_weighted_debt // new_total_debt
```

```
upfront_fee: uint256 = self._calculate_accrued_interest(debt_amount *
avg_interest_rate, self.upfront_interest_period)
```

The original attack shape was:

1. open a large low-rate helper Trove,
2. perform the real fee-bearing action while the average rate is depressed, and
3. close the helper Trove in the same transaction.

The helper Trove temporarily adds a large amount of low-rate debt to `total_debt` and `total_weighted_debt`, lowering the average interest rate used to quote the real upfront fee. After the real action is completed, the helper Trove is closed, leaving only the real Trove with a suppressed fee.

The present mitigation is a same-timestamp guard, described by the code as `"same block"`, because all calls in one transaction execute under the same `block.timestamp`. The guard attempts to block the helper round trip by preventing a Trove from being closed in the same timestamp as opening or adjusting its interest rate:

`trove_manager::close_trove():807`

```
assert convert(trove.last_interest_rate_adj_time, uint256) !=
    block.timestamp, "same block"
```

This blocks the original open-and-close helper pattern because `open_trove()` sets `last_interest_rate_adj_time` to the current timestamp, and `adjust_interest_rate()` also updates it.

However, the guard does not cover `borrow()` on a pre-existing helper Trove.

`borrow()` increases the Trove debt and updates `last_debt_update_time`:

`trove_manager::borrow():591-592`

```
trove.debt = new_debt
trove.last_debt_update_time = convert(block.timestamp, uint64)
```

but it does not update `last_interest_rate_adj_time`.

Therefore, an attacker can use the following bypass:

1. Create a low-rate helper Trove in an earlier block.
2. In a later block, call `borrow()` on the helper Trove with a large amount.
3. The helper debt is added to total_debt and total_weighted_debt, depressing the TroveManager-wide average.
4. In the same block, perform the real fee-bearing action, such as `open_trove()`, `borrow()`, or a premature `adjust_interest_rate()` that charges an upfront fee.
5. The real action receives a lower upfront fee.
6. In the same block, call `close_trove()` on the helper Trove.

Closing succeeds because `last_interest_rate_adj_time` is old, even though the helper debt was increased and the fee basis was manipulated in the same block through `borrow()`.

Since `borrow()` updates `last_debt_update_time` to the current timestamp, the enlarged helper Trove accrues no additional elapsed-time interest before it is closed in the same block, `block.timestamp - convert(trove.last_debt_update_time, uint256)` is zero.

The attacker still repays the pre-existing helper debt, any interest already accrued on that debt before the `borrow()` call, the new helper borrow principal, and the helper

borrow upfront fee. When sufficient idle liquidity is available, the new helper borrow principal is not the main economic cost because the attacker receives those borrowed tokens and can use them toward closing the helper. The meaningful cost is the helper upfront fee, any accrued interest on the small pre-existing helper debt, the small initial setup fee for opening the helper, and any execution friction from obtaining the borrow tokens.

The pre-existing interest cost and setup fee can be made negligible by keeping the helper Trove at minimum debt and minimum rate before the attack block. The large temporary helper debt exists only for the same block, so it depresses the upfront-fee basis without bearing meaningful elapsed-time interest.

The guard uses a timestamp tied to rate changes, but the exploit only requires a same-block debt increase at an already-low rate. Therefore, `last_interest_rate_adj_time` is not an adequate proxy for "this Trove just affected the fee basis".

M2	Deposits can capture accrued Trove interest because Lender share pricing uses stale reported assets	**MITIGATED**

`Lender` is a Yearn `TokenizedStrategy`. Its share price is based on the cached `TokenizedStrategy.totalAssets` value, which is refreshed only when a report runs.

The Lender's live asset value is calculated in `_harvestAndReport()` as:

`Lender::_harvestAndReport():127-130`

```
function _harvestAndReport() internal override returns (uint256){
    // Total assets is whatever idle asset we have + the latest total debt
    // figure from the Trove Manager
    return asset.balanceOf(address(this)) +
        TROVE_MANAGER.sync_total_debt();
```

```
}
```

The TroveManager side accrues aggregate interest over time, but that accrued interest is only materialized into `total_debt` when `sync_total_debt()` or `_sync_total_debt()` is called:

`trove_manager::_sync_total_debt():1467-1479`

```
pending_agg_interest: uint256 = math._ceil_div(
   self.total_weighted_debt *
       (block.timestamp - self.last_debt_update_time),
   _ONE_YEAR * self.borrow_token_precision
)

new_total_debt: uint256 = self.total_debt + pending_agg_interest

self.total_debt = new_total_debt
self.last_debt_update_time = block.timestamp
```

As a result, the Lender's true live assets can increase as Trove interest accrues, while the Yearn share-pricing snapshot remains stale until the next report.

The deposit path mints shares using the currently cached `totalAssets`, before the new deposit is added to accounting:

`TokenizedStrategy::deposit():506,510`

```
shares = _convertToShares(S, assets, Math.Rounding.Down);
_deposit(S, receiver, assets, shares);
```

Therefore, an attacker can deposit immediately before a report and mint shares at a stale PPS that does not include interest already accrued by existing Troves. The

attacker can then trigger or wait for a report. The report calls `Lender._harvestAndReport()`, which calls `TROVE_MANAGER.sync_total_debt()` and recognizes the previously accrued Trove interest as Lender profit.

Although Yearn profit locking prevents the PPS from increasing immediately, it only delays the value transfer. Once the locked profit unlocks, the attacker's freshly minted shares own a pro-rata claim on interest that accrued before the attacker became a Lender shareholder.

Mitigation:
Yearn profit unlocking smooths reported profit over the configured unlock period. A depositor who enters before a report runs remains exposed as a normal Lender shareholder during that period. The issue is therefore treated as report-timing / yield-allocation behavior rather than an atomic extraction path.

M3	Underwater troves in the redemption path can DoS borrows, opens, and lender withdrawals	**BY DESIGN**

When lender idle liquidity is insufficient, `open_trove()`, `borrow()`, and lender withdrawals depend on `TroveManager._redeem()` to free the missing borrow-token `liquidity. _redeem()` walks troves in redemption order and attempts to redeem debt from each eligible trove until the requested shortfall is satisfied.
However, `_redeem()` does not safely handle the case where an eligible redemption target is underwater at the current oracle price. For each target, it computes:

`trove_manager::_redeem():1257-1260`

```
collateral_to_redeem: uint256 = debt_to_free * _PRICE_ORACLE_PRECISION //
    collateral_price

trove_new_collateral: uint256 = trove.collateral - collateral_to_redeem
```

If the traversal reaches an underwater trove before the redemption shortfall has been fully satisfied, collateral_to_redeem can exceed trove.collateral. This causes the call to revert due to checked arithmetic, or due to an explicit collateral sufficiency assertion if one is added.

As a result, the entire borrow, open, or lender withdrawal reverts. No auction is kicked, and `_redeem()` never continues to later healthy troves that may have been able to satisfy the remaining shortfall.

An underwater trove in the redemption path can temporarily block otherwise valid liquidity operations, including new borrows, new troves, and lender withdrawals. This can prevent healthy troves later in the redemption order from being used and can create a denial-of-service condition until the underwater blocker is liquidated or otherwise removed from the redemption path.

Comments:
The team stated that underwater Troves are intended to be handled by liquidation rather than redemption. The reported behavior can block redemption-dependent borrows, opens, or withdrawals until the underwater Trove is liquidated, but this is treated as an accepted liveness tradeoff rather than an unintended security issue.

M4	yv-collateral oracles rely on external price freshness, potentially exposing both lenders and borrowers to stale-price risk	**ACKNOWLEDGED**

The three Yearn-collateral oracles trust the current values returned by their external price inputs. The Chainlink legs use latestAnswer() and only assert that the returned answer is positive. The Yearn pricePerShare() leg is consumed as-is as a vault accounting value.

In `src/oracles/yvweth2_to_usdc_oracle.vy`, the oracle reads the ETH/USD and USDC/USD feeds and the Yearn PPS as follows:

`yvweth2_to_usdc_oracle::get_price():88-100`

```
eth_usd_price: int256 = staticcall _ETH_USD_PRICE_FEED.latestAnswer()
assert eth_usd_price > 0, "wtf"

usdc_usd_price: int256 = staticcall _USDC_USD_PRICE_FEED.latestAnswer()
assert usdc_usd_price > 0, "wtf"

pps: uint256 = staticcall
    IYearnVault(_COLLATERAL_TOKEN.address).pricePerShare()
```

The same pattern appears in `src/oracles/yvcrvusd2_to_usdc_oracle.vy`, which composes CRVUSD/USD, USDC/USD, and Yearn PPS. The `src/oracles/yvusd_to_usdc_oracle.vy` oracle has no Chainlink leg and prices collateral entirely from the current `pricePerShare()` value.

For the two oracles that compose a Chainlink leg with the Yearn leg, the final collateral price is the product of multiple external values accepted as current. Independent staleness windows in either leg compound multiplicatively in the resulting USDC-per-share quote.
The Chainlink deviation thresholds do not bound total oracle error by themselves. Under normal operation they limit ordinary drift in each Chainlink leg, but the final price composes multiple inputs and the Yearn PPS leg can lag independently. A modest stale-high numerator, stale-low denominator, and stale-high PPS can therefore compound into a larger overvaluation than any single feed deviation threshold.

Stale-high direction, lender-side bad debt

In a Yearn V3 Vault, `pricePerShare()` is derived from a cached `totalAssets` value that is refreshed only when a keeper calls `process_report(strategy)` against each backing strategy. Recognition is asymmetric: gains are released gradually via profit unlocking, so PPS rises smoothly between reports, but losses are recognized in a single step at the next report. Between reports, an unreported strategy loss therefore leaves PPS stale-high, with no on-chain signal the oracle can use to detect it. The same shape applies on the Chainlink side: a stale/frozen feed will continue returning that value through `latestAnswer()` until the next valid round.

The inflated price flows directly into the collateral-ratio checks that authorize new risk. `open_trove()`, `borrow()`, and `remove_collateral()` all consume `self.price_oracle.get_price()` and compare the resulting collateral ratio against `minimum_collateral_ratio`.

Let `d` be the fractional amount by which true collateral value is below the oracle value: `true_value = oracle_value * (1 - d)`.

Let `CR_pre` denote the trove's collateral ratio (in oracle terms) immediately before the staleness is corrected.

Since `CR_pre = oracle_value / debt`, the true collateral coverage is: `CR_true = CR_pre * (1 - d)`.
A trove becomes liquidatable when `CR_true < MCR`. Pure collateral shortfall, ignoring liquidation fees, begins when `CR_true < 1`, i.e.:

`d > 1 - 1 / CR_pre`.

This is not Flex's realized lender-loss boundary, however. In Flex's underwater liquidation path the liquidator receives the trove collateral and repays only:

`collateral_value / (1 + liquidation_fee)`,

while the full trove debt is removed from `total_debt`. Realized lender loss therefore begins earlier, at any `CR_true < 1 + liquidation_fee`:

`d > 1 - (1 + liquidation_fee) / CR_pre`.

The realized lender loss per dollar of debt is:

`max(0, 1 - CR_pre * (1 - d) / (1 + liquidation_fee))`.

With an uncapped/default liquidation attempt, the underwater branch activates when `CR_pre * (1 - d) < 1 + liquidation_fee`. Above this threshold the liquidator's fee is paid out of the trove's own collateral surplus and the Lender takes no loss. Below it, the formula gives the per-dollar loss. With `liquidation_fee = 0` the loss expression collapses to the pure collateral-shortfall model `1 - CR_pre * (1 - d)`. `liquidation_fee` is set by linear interpolation between `min_liquidation_fee` and `max_liquidation_fee` based on the trove's collateral ratio at liquidation time, capped at `max_liquidation_fee` when that ratio is at or below `max_penalty_collateral_ratio`. For factory-created markets, any trove that reaches the underwater branch uses `max_liquidation_fee`: the factory requires `max_penalty_collateral_ratio >= 1 + max_liquidation_fee`, and the interpolated-fee region above `max_penalty_collateral_ratio` cannot satisfy the underwater condition.

For the deployed parameters in `script/DeployMarket.s.sol` (`MCR = 1.10`, `max_penalty_collateral_ratio = 1.05`, `max_liquidation_fee = 5%`) the underwater branch activates when `CR_true < 1.05`.

Selected values, using `liquidation_fee = max_liquidation_fee = 5%`:

CR_pre	Loss threshold d	Loss / $1 at d = 0.10	Loss / $1 at d = 0.20
1.10 (= MCR)	~4.5%	~$0.057	~$0.162
1.25	16%	$0	~$0.048
1.50	30%	$0	$0

Aggregate bad debt is therefore not linear in `d` over the whole book. It is a thresholded function applied trove-by-trove against the actual distribution of `CR_pre`. Healthy collateral buffers absorb modest staleness, while the protocol is exposed primarily through the tail of thinly collateralized troves when `d` is large.

Stale-low direction, borrower-side liquidation risk

The same issue applies in the stale-low direction. A feed that latches onto a flash-crash low and does not update cleanly during recovery, or a feed that stops updating, can keep returning a stale-low price. While that staleness persists, troves whose true collateral value is healthy can compute as below-MCR and become liquidatable.

Liquidation in this case is not a protocol-solvency event because the debt is repaid and the Lender is made whole. It is instead a wealth transfer from the borrower, who can lose equity to the liquidator/receiver through the stale valuation and liquidation penalty. The PPS leg can create the same stale-low borrower-side risk when vault gains or loss recoveries are not yet reported. In that case, `pricePerShare()` can understate the live realizable value of the yv collateral, causing healthy troves to appear closer to liquidation or below MCR. The stale-high PPS direction is the more

direct lender bad-debt risk, while stale-low PPS primarily harms borrowers through premature liquidation risk or reduced borrowing capacity.

Affected oracles

- yvWETH-2 / USDC: the oracle is meant to return the USDC value of one yvWETH-2 share. It composes ETH/USD, USDC/USD, and the yvWETH-2 share-to-WETH PPS. A stale-high ETH/USD value, stale-low USDC/USD value, or stale-high PPS overstates collateral value. The reverse directions understate it. Because ETH is volatile, freshness of the Chainlink legs matters even if the Yearn PPS leg is correct.
- yvcrvUSD-2 / USDC: the oracle is meant to return the USDC value of one yvcrvUSD-2 share. It composes CRVUSD/USD, USDC/USD, and the yvcrvUSD-2 PPS. A stale-high CRVUSD/USD value, stale-low USDC/USD value, or stale-high PPS overstates collateral value. The reverse directions understate it. The two Chainlink feeds are both stablecoin/USD feeds, but the ratio can still be wrong if either one is stale, and the PPS leg can be stale independently.
- yvUSD / USDC: the oracle derives the price only from yvUSD PPS, scaled from the collateral token's share decimals into the oracle's expected precision. The tradeoff is that PPS is the entire price input: stale-high PPS overstates collateral value one-for-one.

In all three cases, `pricePerShare()` is vault accounting, not a freshness-checked market price. It can diverge from current realizable value when reports are delayed, strategy losses have not yet been reported, or withdrawals are constrained.

yvWETH-2 / USDC is the highest-risk oracle because stale PPS risk is combined with a volatile ETH/USD leg, so a few percent of overvaluation can matter for thinly collateralized troves. yvcrvUSD-2 / USDC and yvUSD / USDC are lower-volatility cases, but still material in stress scenarios because stablecoin feeds and Yearn PPS can lag too.

This is also a liveness/safety tradeoff. There are several possible ways to handle stale or uncertain oracle inputs, including hard reverts, market pauses, conservative risk parameters in combination with fallback oracles, or explicit risk disclosure. Managing stale inputs is not free: it can preserve pricing safety but may also block trove operations or interfere with liquidations during market stress. Continuing to operate preserves liveness, but accepts valid-but-old Chainlink data and stale Yearn vault accounting as part of the market risk. If the protocol intentionally prioritizes liveness, that choice should be explicit to users and integrators: stale-high inputs can create lender bad debt near the liquidation threshold, while stale-low inputs can make otherwise healthy borrowers liquidatable or reduce their borrowing capacity.

Comments:
The team treats configured oracle failure or stale Chainlink data as immutable market risk, similar to Morpho-style market design. If an oracle becomes unreliable, users are expected to migrate to a new market.

M5	Liquidation receiver callback allows Lender shareholders to redeem at stale PPS before underwater-loss report	**MITIGATED**

A previously reported stale-PPS issue identified that Lender share pricing can become incorrect when real Lender assets change but the Yearn `TokenizedStrategy` cached `totalAssets` snapshot is not refreshed at the right time.

The current code attempts to address the underwater-liquidation side of this issue by forcing a Lender report during underwater Trove liquidation. However, the forced report is executed after a liquidator-controlled receiver callback.

During `liquidate_trove()`, when a Trove is underwater, the function removes the full Trove debt from TroveManager accounting:

`trove_manager::liquidate_trove():1062-1067`

```
self._accrue_interest_and_account_for_trove_change(
    0,
    trove_debt_after_interest,
    0,
    trove_weighted_debt,
)
```

For underwater Troves, the liquidator repays less than the full Trove debt, while the full Trove debt is removed from `total_debt`. This realizes a loss for the Lender vault.
However, after this accounting mutation, the function transfers collateral to the receiver and invokes a receiver-controlled callback when non-empty `data` is provided:

`trove_manager::liquidate_trove():1113`

```
assert           extcall           self.collateral_token.transfer(receiver,
collateral_to_liquidate, default_return_value=True)

if len(data) != 0:
    extcall ITaker(receiver).takeCallback(
        trove_id,
        msg.sender,
        collateral_to_liquidate,
        debt_to_repay,
        data,
    )
```

Only after that callback returns does the function pull the liquidation repayment into the Lender and trigger the forced Lender report:

`trove_manager::liquidate_trove():1127-1134`

```
assert extcall self.borrow_token.transferFrom(msg.sender, self.lender,
debt_to_repay, default_return_value=True)

if is_underwater:
   lender: ILender = ILender(self.lender)
   keeper: IKeeper = IKeeper(staticcall lender.keeper())
   extcall lender.disableHealthCheck()
   extcall keeper.report(lender.address)
```

This creates a callback window where the live asset value used by `_harvestAndReport()` has already decreased, because TroveManager debt was reduced by the full underwater Trove debt, but the Yearn `TokenizedStrategy` cached `totalAssets` value used for `redeem()` has not yet been refreshed.

A liquidator that already holds Lender shares can set the receiver to an attacker contract and pass non-empty callback data. During `takeCallback()`, the attacker calls `lender.redeem()` and exits at the stale pre-loss PPS. The forced report runs only after the callback, so the attacker avoids their pro-rata share of the underwater liquidation loss.

The attack flow is:

1. Attacker holds Lender shares.
2. Attacker liquidates an underwater Trove with receiver = attacker contract and non-empty callback data.
3. liquidate_trove() removes the full underwater Trove debt from total_debt.
4. liquidate_trove() transfers collateral to the attacker receiver.
5. liquidate_trove() calls attacker.takeCallback().
6. Inside the callback, the attacker calls lender.redeem().
7. The redemption uses stale pre-loss Lender PPS because the forced report has not run yet.

8. liquidate_trove() resumes, pulls the liquidation repayment, and finally reports the underwater loss.
9. The attacker has already exited, so the loss share that should have been borne by the attacker is pushed to the remaining Lender shareholders.

This does not rely on calling `report()` from the callback. In fact, a callback-time report can revert under the default Yearn health check because it is a loss report. The validated path is to redeem existing Lender shares before the protocol's own forced report runs.

Mitigation:
Impact is mitigated by the fact that a shareholder can normally exit before voluntarily triggering the liquidation, and by the requirement that the liquidator/receiver also hold Lender shares. The demonstrated callback path was therefore treated as a limited stale-PPS ordering concern rather than a standalone severe issue.

LOW SEVERITY:

ID	Description	STATUS
L1	Unhealthy Troves can adjust interest rate after cooldown and move out of borrow-triggered redemption order	**BY DESIGN**

`TroveManager.adjust_interest_rate()` only checks the Trove's collateral ratio when the adjustment is premature and therefore charges an upfront fee:

`trove_manager::adjust_interest_rate():728,742`

```
if block.timestamp < convert(trove.last_interest_rate_adj_time, uint256) +
        self.interest_rate_adj_cooldown:
    ...
```

```
    assert collateral_ratio >= self.minimum_collateral_ratio,
        "!minimum_collateral_ratio"
```

After `interest_rate_adj_cooldown` has passed, this branch is skipped. The function then updates the Trove's annual interest rate, refreshes debt, and reinserts the Trove in `SortedTroves` without checking whether the Trove is still above `minimum_collateral_ratio`:

`trove_manager::adjust_interest_rate():728-762`

```
if block.timestamp < convert(trove.last_interest_rate_adj_time, uint256) +
        self.interest_rate_adj_cooldown:
    ...

trove.annual_interest_rate = new_annual_interest_rate
trove.last_interest_rate_adj_time = convert(block.timestamp, uint64)

trove.debt = new_debt
trove.last_debt_update_time = convert(block.timestamp, uint64)

extcall self.sorted_troves.re_insert(
   trove_id,
   new_annual_interest_rate,
   prev_id,
   next_id
)
```

As a result, a Trove that has become unhealthy due to price movement or accrued debt can still adjust its rate after cooldown. This lets the borrower move the Trove away from the low-rate tail of `SortedTroves` even though the Trove is already below `minimum_collateral_ratio` and therefore liquidatable.

This matters for borrow-triggered redemptions. When a borrower requests borrow tokens and the Lender does not have enough idle liquidity, `TroveManager._transfer_borrow_tokens()` redeems collateral from other Troves to satisfy the shortfall:

`trove_manager::transfer_borrow_tokens():1511-1517`

```
if amount > available_liquidity:
   ...
   collateral_out: uint256 = self._redeem(amount - available_liquidity,
annual_interest_rate)
```

The internal redemption path only redeems Troves with rates lower than the redeemer's rate:

`trove_manager::_redeem():1220`

```
if not is_zombie_trove and redeemer_annual_interest_rate <=
trove.annual_interest_rate:
    break
```

Therefore, an unhealthy low-rate Trove can raise its rate after cooldown and avoid being selected by borrow-triggered redemptions that would otherwise reach it. The redemption pressure can then be redirected to another lower-rate Trove, including a healthy Trove.

Comments:
The team stated that Troves below the minimum collateral ratio are intended to be handled by liquidation. Although such Troves can still adjust rate after cooldown and change borrow-triggered redemption ordering, they remain liquidatable.

L2	Premature rate-adjustment fee ignores the newly selected interest rate	**DISMISSED**

In `trove_manager`, `adjust_interest_rate()` passes `new_annual_interest_rate` into `_get_upfront_fee(..., True)`, but `_get_upfront_fee()` ignores that parameter when `is_existing_debt=True`.

`trove_manager::_get_upfront_fee():1396-1399`

```
new_total_weighted_debt: uint256 = (
    self.total_weighted_debt
    if is_existing_debt
    else self.total_weighted_debt + (debt_amount * annual_interest_rate)
)

avg_interest_rate = new_total_weighted_debt // new_total_debt
```

As a result, the upfront fee charged for a premature interest-rate adjustment is independent of the new rate selected by the borrower. Raising a Trove to the maximum annual rate and lowering it to a near-minimum annual rate can produce the same upfront fee, assuming the same Trove debt and global system state.

The March 2026 audit already included finding 2.8.6 “Upfront fee weighted debt calculation does not account for accrued interest”, which was acknowledged. This issue is another side of the same underlying problem: the upfront-fee preview does not mirror the weighted-debt accounting that is later applied by `_accrue_interest_and_account_for_trove_change()`.

Comments:
The team considers the premature rate-adjustment fee sufficient as a cost for frequent rate changes. The narrower observation is that `new_annual_interest_rate` does not affect `_get_upfront_fee(..., is_existing_debt=True)`, so the fee is effectively based on the branch average rather than the selected new rate. If the intended model is a flat branch-average fee for any premature adjustment, this is a fee-model/documentation issue.

OTHER / ADVISORY ISSUES:

This section details issues that are not thought to directly affect the functionality of the project, but we recommend considering them.

ID	Description	STATUS
A1	`minimum_debt` deployment parameter is documented as scaled but interpreted as unscaled	**RESOLVED**

`Factory.DeployParams.minimum_debt` is documented as though the deployer should pass a value already scaled by the borrow token precision:

`factory::DeployParams:42`

```
minimum_debt: uint256  # minimum borrowable amount, e.g., `500 * borrow_token_precision` for 500 tokens
```

However, `TroveManager.initialize()` multiplies the provided value by `borrow_token_precision` again:

`trove_manager::initialize():223,231,232`

```
borrow_token_precision: uint256 = 10 ** convert(
    staticcall IERC20Detailed(params.borrow_token).decimals(),
    uint256
)

self.borrow_token_precision = borrow_token_precision
self.min_debt = params.minimum_debt * borrow_token_precision
```

Therefore, the code expects `minimum_debt` to be passed as an unscaled whole-token amount.

The repository's tests and scripts confirm this intended usage:

In the base test fixture, the value is defined as: `uint256 public minimumDebt = 500; // 500 tokens`. The market deployment passes the unscaled value: `minimum_debt: 500`. The test suite then expects the stored value to be scaled by `TroveManager`: `assertEq(troveManager.min_debt(), minimumDebt * BORROW_TOKEN_PRECISION, "E8")`.

So the implementation is internally consistent, but the Factory parameter documentation is wrong. If a deployer follows the Factory comment, a mismatch will be created, that leads to higher impact, if the borrow token has high decimals. This can make normal borrowing impossible or force extremely large minimum positions.

A2	Registry uses order-independent pair keys for directional markets	**BY DESIGN**

The Registry indexes markets by a pair key computed with XOR:

`registry::_get_pair_key():213-222`

```
@internal
@pure
def _get_pair_key(a: address, b: address) -> uint256:
    …
    return convert(a, uint256) ^ convert(b, uint256)
```

This makes the key order-independent:

`_get_pair_key(tokenA, tokenB) == _get_pair_key(tokenB, tokenA)`

However, Flex markets are directional. A market with `collateral token = tokenA` and `borrow token = tokenB` has a different economic meaning that a market with `collateral token = tokenB` and `borrow token = tokenA`. In the first market, users deposit tokenA and borrow tokenB. In the second market, users deposit tokenB and borrow tokenA.

Because the Registry uses an order-independent key, both market directions are stored under the same markets_by_pair bucket. As a result, calls such as:

```
get_all_markets_for_pair(tokenA, tokenB)
find_market_for_pair(tokenA, tokenB, index)
markets_count_for_pair(tokenA, tokenB)
```

can return markets for both directions.

The XOR key is also structurally collision-prone because different token pairs can satisfy: `A ^ B == C ^ D`.

In practice, the more realistic concern is not arbitrary collision construction, since endorsed markets are controlled by the Registry owner. The primary issue is that inverse directional markets are intentionally collapsed into the same bucket. Integrators, or frontends that assume `get_all_markets_for_pair(collateral, borrow)` returns only markets with that exact direction may surface or select the wrong market. This can lead to incorrect UI results or mistakes, for example showing a USDC → WETH market when the caller requested a WETH → USDC market.

Comments:

The team confirmed that the Registry intentionally groups both directions of a token pair, so callers may discover both long and short markets from the same unordered key.

A3	Registry market getters continue returning unendorsed markets	**BY DESIGN**

Registry.unendorse() only updates market_status:

registry::unendorse():187-203

```
@external
def unendorse(trove_manager: address):
    assert msg.sender == DADDY, "bad daddy"
    assert self.market_status[trove_manager] == Status.ENDORSED,
        "!ENDORSED"

    self.market_status[trove_manager] = Status.UNENDORSED

    log UnendorseMarket(trove_manager=trove_manager)
```

It does not remove the market from the append-only market arrays:

registry:56,58

```
markets: public(DynArray[address, _MAX_UINT32])
markets_by_pair: HashMap[uint256, DynArray[address, _MAX_UINT32]]
```

As a result, the following getters continue returning unendorsed markets:

- `get_all_markets()`
- `get_all_markets_for_pair()`
- `find_market_for_pair()`
- `markets_count()`
- `markets_count_for_pair()`

This may be intentional because markets are documented in storage as an append-only list. However, the contract title and notice describe the Registry as storing and managing endorsed market addresses, and several getter names can be interpreted as returning active endorsed markets.

Frontends or integrations that rely on the Registry getters without separately checking market_status may continue to display, route to, or otherwise treat unendorsed markets as endorsed. This can create integration and user-facing risk if unendorsement is intended to signal that a market should no longer be trusted, recommended, or used.

Comments:
The team confirmed that Registry market arrays are intended to be historical append-only lists.

A4	Deployment parameters are only specified for one of the three intended Yearn collateral markets	**BY DESIGN**

The repository contains oracle contracts for three Yearn collateral markets:

- yvUSD / USDC
- yvWETH-2 / USDC
- yvcrvUSD-2 / USDC

However, the concrete market deployment parameters are only specified for the yvUSD/USDC market in `script/DeployMarket.s.sol`. No per-market `DeployParams` manifests exist for yvWETH-2/USDC and yvcrvUSD-2/USDC at the time of the audit.

These parameters are expected to be market-specific and security-relevant, and include:

- minimum debt,
- collateral ratio thresholds,
- liquidation fee bounds,
- upfront fee timing,
- auction start, minimum, and re-kick buffers,
- auction decay speed and length.

The yvWETH-2/USDC and yvcrvUSD-2/USDC markets have different oracle-deviation, liquidity, redemption, and vault-reporting assumptions from yvUSD/USDC, so the absence of explicit parameters makes it harder to verify that the intended safety margins are preserved across all three markets.

Comments:
The team stated that only the yvUSD market is planned for deployment currently, while the other Yearn collateral markets are older candidates and are not expected to be used soon.

A5	Inefficiency in ERC4626 `withdraw` and `redeem` when `maxLoss` is not 100% and there is not enough idle liquidity	**ACKNOWLEDGED**

`Lender` provides ERC4626 `withdraw` and `redeem` functions, which rely on `TokenizedStrategy._withdraw`. When not enough idle liquidity is present to satisfy the request, the system attempts to free funds, by calling `Lender._freeFunds`:

`TokenizedStrategy::_withdraw():1008-1032`

```
uint256 idle = _asset.balanceOf(address(this));
uint256 loss;
// Check if we need to withdraw funds.
if (idle < assets) {
    // Tell Strategy to free what we need.
    unchecked {
```

```
        IBaseStrategy(address(this)).freeFunds(assets - idle);
    }

    // Return the actual amount withdrawn. Adjust for potential under
    // withdraws.
    idle = _asset.balanceOf(address(this));

    // If we didn't get enough out then we have a loss.
    if (idle < assets) {
        unchecked {
            loss = assets - idle;
        }
        // If a non-default max loss parameter was set.
        if (maxLoss < MAX_BPS) {
            // Make sure we are within the acceptable range.
            require(
                loss <= (assets * maxLoss) / MAX_BPS,
                "too much loss"
            );
        }
```

`Lender._freeFunds` calls `trove_manager.redeem`, kicking off an auction to free up the required collateral.
However, when this flow is taken, if `maxLoss != 100%`, `TokenizedStrategy` checks whether enough liquidity has been freed up that satisfies the request, minus the loss. For Flex, no liquidity is ever freed up in the same transaction, and thus the call results in a revert, with an extra wasteful call to `Lender._freeFunds`.

In summary, a call to `Lender.withdraw` and `Lender.redeem` with `maxLoss != 100%` when there is not enough idle liquidity will always result in an unneeded call to `Lender._freeFunds`, since the transaction will revert immediately after.

DISCLAIMER

The audited contracts have been analyzed using automated techniques and extensive human inspection in accordance with state-of-the-art practices as of the date of this report. The audit makes no statements or warranties on the security of the code. On its own, it cannot be considered a sufficient assessment of the correctness of the contract. While we have conducted an analysis to the best of our ability, it is our recommendation for high-value contracts to commission several independent audits, a public bug bounty program, as well as continuous security auditing and monitoring through Dedaub Security Suite.

ABOUT DEDAUB

Dedaub offers significant security expertise combined with cutting-edge program analysis technology to secure some of the most prominent protocols in DeFi. The founders, as well as many of Dedaub's auditors, have a strong academic research background together with a real-world hacker mentality to secure code. Protocol blockchain developers hire us for our foundational analysis tools and deep expertise in program analysis, reverse engineering, DeFi exploits, cryptography and financial mathematics.